

single_backtest_runner

February 17, 2026

1 Single Backtest Runner

This notebook orchestrates a single backtest run using the `ggTrader` orchestrator api. It supports flexible configuration and interactive visualization of performance results.

```
[9]: import sys
import os
import pandas as pd
import numpy as np
import vectorbt as vbt
import matplotlib.pyplot as plt

# Auto-reload custom modules
%load_ext autoreload
%autoreload 2

# Ensure project root is in path
project_root = os.path.abspath(os.path.join(os.getcwd(), '..', 'src'))
if project_root not in sys.path:
    sys.path.append(project_root)

from ggTrader.core.orchestrator import run_backtest_orchestrator

print("Environment initialized.")
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
Environment initialized.
```

```
[10]: # --- Configuration ---
CONSTANTS = {
    "SYMBOLS": None,
    "SYMBOLS_FILE": os.path.join(os.getcwd(), "..", "data", "top_20_USD_1095_movers.json"),
    "INTERVAL": "4h",
    "START_DATE": "2023-01-01",
    "END_DATE": "2025-12-31",
    "START_CASH": 1000,
```

```

    "PORTFOLIO_SHARE": 0.10,
    "FEES": 0.004,
    "USE_MOVERS": 0,
    "DEFAULT_PARAMS": {
        "adx_threshold": 30,
        "adx_length": 30,
        "sar_acceleration": 0.02,
        "sar_maximum": 0.2,
        "atr_multiplier": 1.0,
        "atr_length": 10,
        "use_dmp_cross": True,
    }
}

print("Configuration loaded.")

```

Configuration loaded.

```

[11]: # --- Run Backtest ---
# Set save_results=False for pure interactive work, or True to persist to
↳ results/ folder
results = run_backtest_orchestrator(
    config=CONSTANTS,
    params=CONSTANTS["DEFAULT_PARAMS"],
    save_results=False,
    show_progress=True
)

pf = results["portfolio"]
stats = results["stats"]

print("\nBacktest Complete.")

```

Loading data...

Running backtest...

0%| | 0/1 [00:00<?, ?it/s]

Backtest Complete.

```

[12]: from tabulate import tabulate

# --- Visualization ---
print("Global Portfolio Stats:")
# Convert stats to a DataFrame for a cleaner table view
stats_df = pf.stats().to_frame(name="Portfolio Stats")

```

```
print(stats_df.round(2))
```

Global Portfolio Stats:

	Portfolio Stats
Start	2023-01-01 00:00:00+00:00
End	2025-12-30 16:00:00+00:00
Period	1094 days 20:00:00
Start Value	1000.0
End Value	1918.330415
Total Return [%]	91.833041
Benchmark Return [%]	210.276909
Max Gross Exposure [%]	76.745328
Total Fees Paid	373.330719
Max Drawdown [%]	23.700748
Max Drawdown Duration	389 days 00:00:00
Total Trades	499
Total Closed Trades	495
Total Open Trades	4
Open Trade PnL	-198.876251
Win Rate [%]	39.79798
Best Trade [%]	121.037925
Worst Trade [%]	-63.025613
Avg Winning Trade [%]	15.047161
Avg Losing Trade [%]	-5.108628
Avg Winning Trade Duration	15 days 01:59:23.451776649
Avg Losing Trade Duration	10 days 11:01:12.483221476
Profit Factor	1.718969
Expectancy	2.256983
Sharpe Ratio	1.019053
Calmar Ratio	1.023485
Omega Ratio	1.07154
Sortino Ratio	1.554393

```
[15]: pf.plot(subplots=['drawdowns', 'value']).show()
```

```
[14]: # print(pf.total_profit(group_by=False))
```